

Ray Tracing

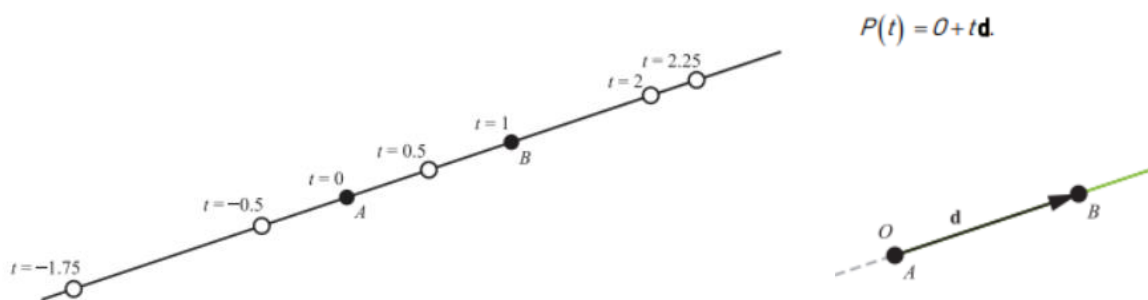
Malek.Bengougam@gmail.com

Partie 1 - Le lancer de rayons en détail

Rayon

Le cœur d'un Ray Tracer est bien évidemment le rayon. Un rayon est un segment de droite (3D) paramétrique ayant une origine (un point) et une direction (un vecteur normalisé).

Paramétrique signifie ici qu'un paramètre ayant la forme d'une variable ('t' par ex.) nous permet de calculer des valeurs (de points) intermédiaires en faisant varier le facteur entre 0 et 1 par exemple.



Il est théoriquement possible d'utiliser des valeurs négatives mais dans notre cas seules les valeurs positives nous intéresseront.

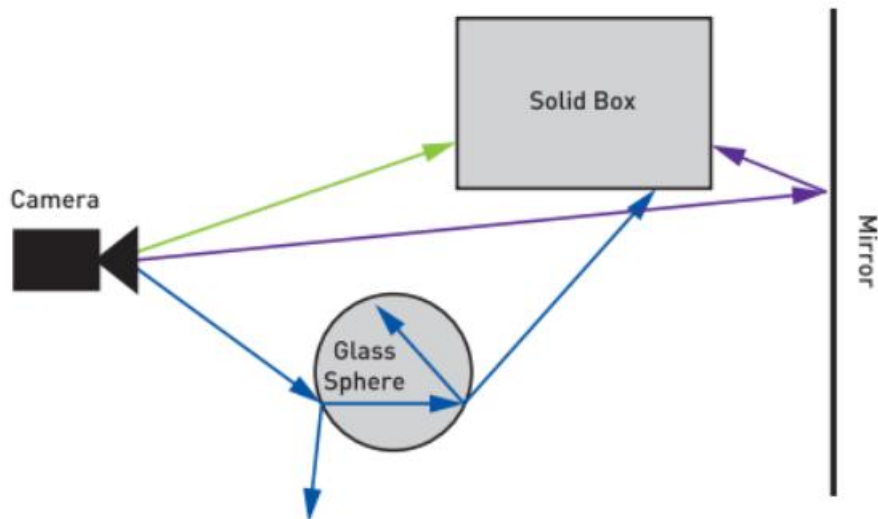
Note : pour diverses raisons (simplicité, optimisations...) certains rayons que nous allons utiliser n'ont pas une direction normalisée.

Intersection

L'intersection d'un rayon avec un objet s'effectue par la résolution d'une équation mathématique. Nous allons nous limiter aux surfaces explicites -càd dont l'équation est soluble de manière analytique- telles que les sphères, les boîtes alignées (AABB, Axis-Aligned Bounding Box) et les triangles.

Les rayons sont émis pour chaque pixel composant une image en tenant compte des paramètres de projection et de caméra.

Un rayon peut intersecter ou rater (*miss*) les objets de la scène. Dans le cas où le rayon rate, on attribue simplement la couleur d'arrière-plan (qui peut éventuellement être facteur de la position).



Il est souvent utile de stocker les valeurs d'intersections min et max d'un rayon. Cela permet tout à la fois de limiter l'action du rayon –dans le cadre d'un rayon paramètre on peut considérer que $t=0$ implique la valeur min, et $t=1$ implique la valeur max- et en quelque sorte d'algorithme de tri en conservant la valeur max d'une intersection si et seulement si cette dernière est la plus proche de la caméra.

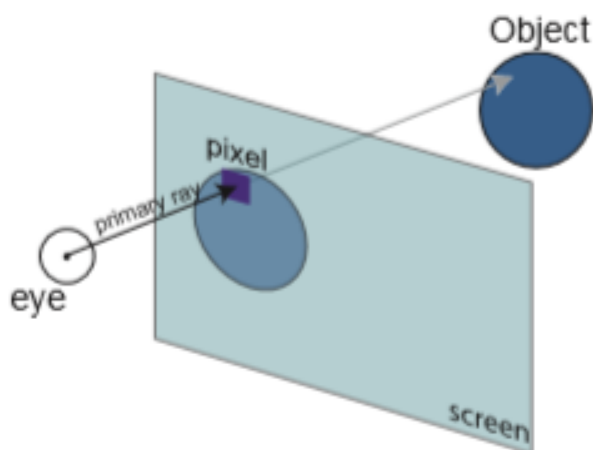
Autre point important, du fait des limites de précision de la représentation nombres réels, particulièrement avec des réels sur 32 bits (float), il est fondamental de prendre en compte la robustesse dans nos algorithmes d'intersections.

L'étape la plus basique consiste à toujours ajouter un epsilon afin de prévenir des cas de figure telles que la "*catastrophic cancellation*", pertes de précision, ou l'accumulation d'erreurs.

Méthode d'Appel

Cette méthode est appelée "ray casting" car les rayons sont envoyés d'une source (caméra) pour percevoir l'environnement.

Comme décrit au paragraphe précédent, la couleur d'un pixel de l'image dépend de l'intersection ou non du rayon.



Lancer de rayons récursifs (méthode de Whitted)

Dans la méthode de Turner Whitted lorsqu'un rayon touche un objet, d'autres rayons (un rebond) sont générés en suivant les lois de réflexions et réfractions de la lumière.

Les rayons sont ainsi divisés en rayons **primaires** –ceux partant de la caméra comme dans la méthode d'Appel- et rayons **secondaires**.

La méthode de Whitted innove également par l'usage de rayons spéciaux comme les détecteurs d'ombre (*shadow feelers*) qui sont des simples *ray cast*, il n'est pas nécessaire de savoir quelle est l'intersection et/ou s'il s'agit de la plus proche, mais juste de savoir s'il y'a un obstacle.

Comme avec tout ce qui est récursif en informatique, il est préférable de mettre une limite à la récursivité. On peut voir chaque rayon généré comme une branche d'un arbre ce qui conduit à nommer cette limite "profondeur (*depth*)".

On arrête donc le processus lorsque la profondeur maximale est atteinte ou lorsque le rayon rate les objets de la scène.

Cette méthode permet de mettre en place une illumination locale et globale car chaque rebond peut générer deux rayons (en fonction du matériau) : un rayon **spéculaire** et un rayon **transmissif**.

On peut également déduire de cette méthode une vision claire du trajet de lumière, ce que l'on appelle le "**light transport**". La méthode de Whitted est la forme de transport la plus brute mais aussi la plus limitée car une image réaliste nécessiterait une infinité de rebond.

Cependant cette méthode n'est pas suffisante pour simuler des matériaux plus complexes que les surfaces parfaitement lisses. D'autre part, la séparation d'un rayon en deux rayons n'est pas non plus correcte d'un point de vue physique.

A ce titre, la méthode de Whitted n'est donc que partiellement réaliste car elle permet de représenter l'illumination globale de façon correcte mais est limitée aux surfaces lisses, tandis que la méthode d'illumination directe de Phong ne permet pas de gérer l'illumination globale mais propose une illumination locale très acceptable (mais non plausible d'un point de vue physique).

En pratique, il est assez habituel de combiner l'équation de Phong avec la méthode Whitted dans le cadre de l'illumination locale, ce qui permet également d'appliquer les méthodes de rendu classiques comme le fait de simuler des surfaces détaillées avec la méthode du bump mapping.

Couleur

Dans la méthode de Whitted, la couleur d'un pixel de l'image est alors déterminée par une accumulation des couleurs à chaque intersection, il est possible que certaines intensités lumineuses implique un dépassement du rang habituel des couleurs [0;1].

Il est recommandé de procéder à une opération de saturation (clamping) afin de ne pas écrire des valeurs trop grandes. Ceci n'est nécessaire que dans la phase de "présentation" c'est-à-dire la visualisation sur un écran ou le stockage sur disque au format image.

En effet, sur un écran LDR (low dynamic range) nous sommes limités à 8 bits par composantes, ce qui implique que toute valeur supérieure à 255 risque de boucler (modulo 256) et donc donner une couleur incorrecte.

Il est souvent souhaitable de gérer le rendu intermédiaire dans une précision haute (à l'aide de couleur représentée par des réels) afin de maintenir un dynamic range le plus large possible.

Correction gamma

Les couleurs que nous allons manipuler vont être purement linéaires (en nombre réel également). Cependant nous savons que les moniteurs ont une représentation limitée du spectre lumineux sous la forme du tristimulus RGB en LDR 8 bits comme on l'a dit.

Il est donc important de corriger les couleurs en appliquant une correction / compression gamma afin de pouvoir afficher et/ou stocker les couleurs en RGB (8 bits par composantes pour nous).

Les images (textures) échantillonnées doivent quant à elles subir une décompression gamma ; pour cela on peut par exemple mettre chaque composante RGB à la puissance 2.2.

Gamma vers linéaire $y = \text{pow}(x, 2.2)$

Linéaire vers gamma $x = \text{pow}(y, 1.0/2.2)$

Note : 2.2 est ici une approximation de la fonction de transfert sRGB qui est plus complexe cf. [https://en.wikipedia.org/wiki/SRGB#The_sRGB_transfer_function_\(%22gamma%22\)](https://en.wikipedia.org/wiki/SRGB#The_sRGB_transfer_function_(%22gamma%22))

Color Buffer

L'avantage du Ray Tracing est qu'il n'est pas nécessaire d'avoir d'algorithme de tri des fragments, car pour chaque pixel on a la garantie d'obtenir la couleur des pixels les plus proches (ou une combinaison dans le cas de la transparence).

On va donc simplement définir une zone mémoire composée de $M*N$ couleurs définissant notre image. Il peut être utile de différencier l'image avant compression gamma (en nombre réels) et l'image générée après la correction gamma (en supposant 8 bits par composantes).

En dernière étape, on appliquera donc une correction linéaire vers gamma notamment avant le stockage de l'image.

Une image

Pour l'image finale nous pouvons utiliser n'importe quel format tels jpeg, png, etc.. en utilisant la bibliothèque STB par exemple.

J'ai ici opté pour le TGA car c'est un format simple et facile à mettre en œuvre notamment dans le cadre de la sauvegarde (écriture d'un entête suivi des données brutes).

La bibliothèque STB reste cependant utile pour charger des images à utiliser comme texture, y compris des images au format HDR.

TP 1 : utiliser une image panoramique (dite "lat-long", latitude-longitude, ou équi-rectangulaire) pour afficher l'arrière-plan de la scène. Une image panoramique s'échantillonne à l'aide de coordonnées polaires où theta représente l'axe longitudinale (right) et phi la latitude (up). La conversion cartésienne vers polaire permettant de générer les coordonnées de texture (theta, phi) peut s'exprimer ainsi :

```
float theta = acos(ray.direction.y) / -PI;
```

```
float phi = atan2(ray.direction.x, -ray.direction.z) / -PI * 0.5f;
```


Partie 2 - Implémentation

Intersection rayon - sphère

Une sphère se représente sous la forme d'une structure composée d'une origine et d'un rayon.

Il est possible de convertir cette structure en équation en établissant le fait que l'intérieur de la sphère (le volume) est un sous-espace positif, et donc négatif en dehors.

Formulé plus simplement, un point compose la sphère si la distance entre ce point et le centre de la sphère est inférieure au rayon de la sphère :

$$\text{sqrt}((x - p).(x - p)) \leq r$$

On peut améliorer cette équation en mettant les deux parties au carré, ce qui donne :

$$(x - p).(x - p) \leq r^2$$

L'intersection peut être calculée par la résolution d'une équation paramétrique du second degré de type at^2+bt+c en remplaçant p dans l'équation par le rayon.

$$(x - (o + td)).(x - (o + td)) - r^2 = 0$$

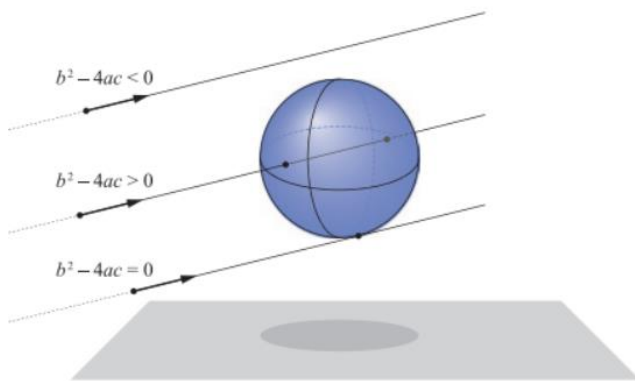
$$\Leftrightarrow ((x - o) + td).((x - o) + td) - r^2 = 0$$

$$\Leftrightarrow (x - o)^2 + 2((x - o).td) + (td)^2 - r^2 = 0$$

$$\Leftrightarrow (d.d)t^2 + 2((x - o).d))t + (x - o).(x - o) - r^2 = 0$$

En posant $f = (x-o)$ on peut déduire $a = d.d$ $b = 2(f.d)$ $c = f.f-r^2$

Le discriminant b^2-4ac nous renseigne alors sur l'état de l'intersection.



Intersection rayon-plan

Un plan est défini par une normale dont la direction indique la partie positive de l'hyper-espace séparé par le plan et une distance à l'origine. On peut également représenter le plan sous la forme d'une équation $ax + by + cz = d$; où (a, b, c) sont les composantes de la normale du plan, d est la distance à l'origine, et (x, y, z) sont les composantes d'un point sur le plan.

Plus pratique pour nous, la forme $n.(p - o) = 0$ où n est la normale, p un point arbitraire sur le plan et o l'origine du plan.

Comme précédemment, on insère l'équation du rayon dans l'équation du plan, à la place de p ici.

Intersection rayon-triangle

L'algorithme le plus utilisé et un des plus performant est l'algorithme de Möller-Trumbore.

TP 2: implémenter l'algorithme de Möller-Trumbore afin d'afficher un maillage 3D

Génération des rayons

On suppose un repère main-gauche, forward = +z pointant VERS l'écran, en espace caméra.
Comme en OpenGL nous allons définir un plan de projection, exprimé par un field-of-view ou une distance focale, sur lequel les rayons émis depuis le point de vue de la caméra vont se projeter.

Selon que l'on souhaite avoir un rendu orthographique ou perspective on va définir une origine et une direction pour chacun des rayons émis à chaque pixel de l'image.

La taille d'un pixel dépend de la résolution de l'écran/image. Nous allons positionner la caméra au centre de l'écran, ce qui implique un repère orthocentré allant de $(-width/2, -height/2)$ à $(width/2, height/2)$. Afin que le lancer de rayon reste indépendant de la résolution, nous allons plutôt "orthonormer" le repère.

Nous allons également supposer que le rayon passe par le centre du pixel : on ajoute donc un demi-pixel aux coordonnées écran pour spécifier le centre.

Pour un pixel (i,j) de l'écran, les coordonnées orthonormées en repère caméra (x, y) sont :

```
x = 2 * (i - (width/2) + 0.5) / width
    = 2 * (i - 0.5 * (width - 1)) / width
y = 2 * (j - (height/2) + 0.5) / height
    = 2 * (j - 0.5 * (height - 1)) / height
```

Cependant, il est courant que les dimensions width et height soient différentes il faut donc compenser la coordonnée x par l'ajout d'un facteur dit "aspect ratio" ratio de width / height

```
aspect_ratio = width / height
x *= aspect_ratio
```

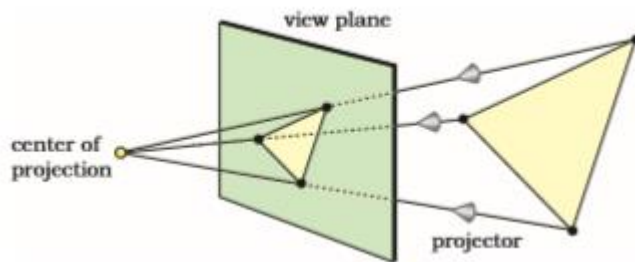
De plus, l'axe y est inversé entre le repère mathématique et le repère de l'écran en pixel. Il suffit de multiplier y par -1 afin d'inverser l'axe.

Projection orthographique

Dans le cas de la projection orthographique la direction d'un rayon est toujours le forward $(0,0,+1)$ pour un repère main-gauche.

L'origine est simplement la position de chaque pixel (x, y, z) où z est ici la distance entre la caméra et le plan de projection.

Projection perspective



Cette fois-ci tous les rayons ont pour origine le centre de projection, et la direction du rayon va du centre de projection vers nos pixels sur le plan de projection (view plane).

Un modèle plus générique

Nous avons déjà la possibilité de spécifier une position arbitraire pour le centre de projection ainsi que pour le plan de projection.

Idéalement, il faudrait également gérer une direction cible pour la caméra, ainsi qu'une orientation arbitraire de manière sensiblement identique à la caméra virtuelle "trou d'épingle (*pinhole*)" que nous avons rencontré en OpenGL.

Nous allons construire une base orthonormée (*Orthonormal Basis*, ONB) de manière similaire à la fonction "LookAt" générant l'équivalent d'une matrice World pour la caméra. Attention ne confondez pas cette LookAt avec la transformation World-View exprimée par une View Matrix qui est l'inverse de la LookAt que nous allons utiliser ici.

Représentation de la scène

Notre scène peut être vue comme un tableau ou liste de différentes primitives. La façon la plus simple au premier abord de gérer nos primitives consiste à raisonner en POO en exploitant le polymorphisme via une interface abstraite "Primitive" et en faisant hériter toutes nos primitives de cette classe.

De même, il nous faut un moyen de conserver l'état d'un rayon (payload) avec des variables telles qu'une référence (pointeur) vers la Primitive la plus proche du rayon ainsi que les propriétés de l'intersection (distance, matériaux, position, normale etc..., certaines de ces propriétés pouvant éventuellement être calculées plus tardivement).

Partie 3 - améliorations visuelles

Ombres

On l'a dit plus haut, Whitted a introduit la notion de "*shadow feelers*" c'est-à-dire un lancer de rayon direct vers les lumières afin de détecter si un fragment est masqué par un objet situé entre le fragment et la lumière.

On va donc ajouter une fonction de visibilité exécutée à chaque rebond et qui nous indique si le fragment est ombré, dans ce cas inutile de calculer l'illumination directe, ou non.

Anticrénelage (antialiasing)

Méthode du super-sampling

Plutôt que d'envoyer un seul rayon par pixel, la technique du super-sampling consiste à subdiviser chaque pixel en $n \times n$ sous-pixels pour lesquels on génère un rayon.

Ceci revient à augmenter artificiellement la résolution de l'image, chaque pixel étant alors calculé comme la moyenne des sous-pixels générés par le super-sampling.

Au moment de la génération des rayons, on va donc émettre $n \times n$ rayons (par exemple $2 \times 2 = 4$ rayons au lieu d'un) et chaque rayon est orienté vers le centre de chacun des sous-pixel de rayon $1/(2*n)$.

Perturbation aléatoire (jittering)

Le fait d'augmenter la qualité de l'image via le super-sampling implique une surcharge de travail car le nombre de rayons générés croît de manière exponentielle à chaque subdivision.

Intuitivement, si l'on regarde une photo, le crénelage n'apparaît pas car les bordures des objets sont mélangées avec l'environnement immédiat ou distant (background).

La technique du *jittering* consiste à introduire une perturbation pseudo-aléatoire du rayon de sorte que les bordures des intersections laissent apparaître une contribution de l'arrière-plan du fait que le rayon est dévié de manière aléatoire.

Il faut prêter attention au type de distribution en évitant les distributions ayant une trop large variance, ce qui produit du bruit "blanc" (white noise) comme sur les réseaux hertziens. En C++ moderne on dispose par exemple d'une distribution pseudo-aléatoire uniforme dans le code suivant

```
#include <functional>
#include <random>
inline float random_float() {
    static std::uniform_real_distribution< float > distribution(0.0f, 1.0f);
    static std::mt19937 generator;
    static std::function< float ()> rand_generator = std::bind(distribution, generator);
    return rand_generator();
}
```

La génération de nombres réels pseudo-aléatoires est très utile pour diverses techniques permettant de dépassant le rendu trop lisse de la méthode de Whitted, par exemple en perturbant les rayons *shadow-feelers*, en perturbant l'origine des rayons (simulation de lentille, depth-of-field), en ajoutant un délai aléatoire lors de la génération d'un rayon (motion blur) etc...

Textures

Vous pouvez reprendre le code des TPs précédent basés sur la bibliothèque STB pour ce qui concerne le chargement des images bitmaps en mémoire.

Afin de plaquer des textures sur nos objets il nous faut en premier lieu générer des coordonnées de texture dans le cas où nos primitives n'en disposent pas.

Cependant les primitives que nous avons rencontrées jusqu'à présent sont essentiellement procédurale ou semi-procédurale, telle notre sphère.

Dans ce cas il nous faut calculer des coordonnées de texture manuellement à chaque intersection du rayon avec la primitive.

Planaire

Sphérique

Cylindrique

Triangulaire (barycentrique)

Matériaux

Illumination directe

Afin de calculer l'illumination directe il nous faut au préalable disposer au moins d'une direction vers une lumière ponctuelle (directionnelle, positionnelle ou spot) ainsi qu'une normale au point d'intersection.

La normale dépendant de la surface elle peut être calculée en fonction du type de primitive intersecté. Dans le cas d'une sphère, on peut calculer la normale comme un vecteur normalisé produit par la différence entre le point d'intersection et le centre de la sphère.

$N = (P - sphere.position).normalize()$

Cette normale va également nous servir à calculer les réflexions et réfractions de l'illumination indirecte (aussi appelée illumination globale, GI).

Dans le cas d'un triangle, la normale peut être calculée à l'aide des coordonnées barycentriques (déjà calculées dans l'algorithme d'intersection rayon-triangle) qui vont nous permettre d'interpoler et pondérer les normales des 3 sommets d'un triangle.

Matériau diffus

La loi du cosinus de Lambert nous permet de simuler une réflexion diffuse "lambertienne" c'est-à-dire en considérant une surface parfaitement lisse.

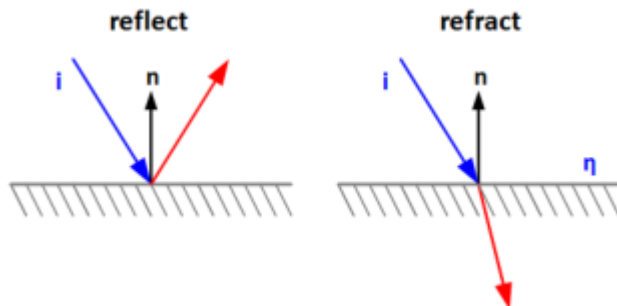
C'est généralement suffisant pour donner un semblant de réalisme mais il n'est pas possible dans ce cas de simuler les objets brillants (glossy).

On peut éventuellement implémenter l'équation d'illumination de Phong (ou Blinn-Phong) afin de donner un aspect brillant mais comme il s'agit ici d'illumination directe il ne nous sera pas possible de gérer des réflexions entre les objets.

Illumination indirecte

Tout l'intérêt du ray-tracing vient de la faciliter à exploiter les inter-réflexions. On a ici les mêmes limites qu'en illumination directe (seules les surfaces parfaitement lisses peuvent être simulées avec la méthode de Whitted).

Nous allons avoir besoin d'outils mathématiques supplémentaires, à commencer par la fonction *reflect*, déjà utilisée par Phong, et la fonction *refract*.



Matériau spéculaire

Théoriquement, il faudrait séparer les matériaux dits conducteurs et les matériaux isolants aussi appelés matériaux diélectriques.

Les matériaux conducteurs sont essentiellement réfléchissants c'est-à-dire qu'ils réfléchissent la lumière dans des proportions élevées - théoriquement les conducteurs sont tellement réfléchissants qu'ils n'ont pas de réflexion diffuse directe (entre autres parce que la lumière est absorbée et transformée / diffusée sous une autre forme – dissipation de chaleur par ex.).

Parmi les matériaux conducteurs on retrouve essentiellement des matériaux métalliques, tels les métaux polis type chromes que l'on se propose de simuler avec la méthode de Whitted.

Encore une fois, la technique de ray-tracing ne nous permet pas de simuler les réflexions rugueuses on doit donc se contenter des miroirs parfaits.

Matériau transparent

Parmi les matériaux diélectriques (ou isolants) on retrouve à la fois les matériaux opaques de type plastique –qu'ils soient mates ou brillants- ainsi que les matériaux transparents.

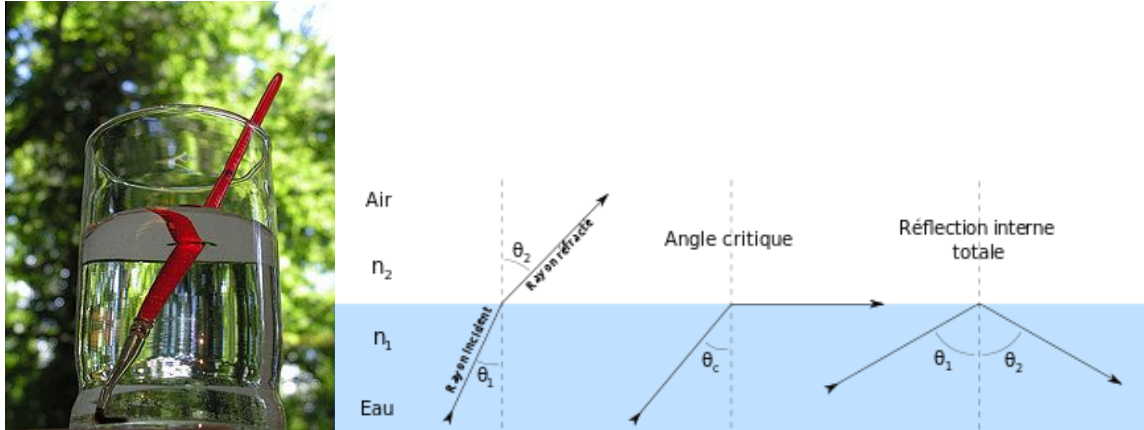
Ils sont caractérisés par le fait d'être à la fois réfléchissants et transmissifs, c'est-à-dire que l'on a à la fois un phénomène de réflexion et de réfraction.

Dans la méthode de Whitted, on génère donc deux rayons, un rayon réfléchi et un rayon réfracté. Ceci n'est pas réaliste, un photon ne se scinde pas en deux, mais permet encore une fois de faire illusion.

La réfraction est un phénomène conduisant un rayon à traverser différents milieux, tels que l'air et l'eau par exemple. Ces milieux sont séparés par une couche parallèle à la surface que l'on appelle interface ou dioptré. La réfraction est décrite d'un point de vue optique par les lois de réfractions de Ibn Sahl – Snell – Descartes.

Il y'a un cas particulier dans le cadre de la réfraction qui apparaît lorsqu'un rayon franchit une interface en passant d'un milieu ayant un indice de réfraction plus grand que celui du milieu d'arrivée, exemple de l'eau (~1.3) vers l'air (~1.0).

Cela se produit à partir d'un angle particulier appelé angle de Brewster et produit un phénomène dit de "réflexion totale (total internal reflection)", et donne, en reprenant le cas eau – air, l'impression que l'interface est totalement réflexive, comme un miroir. C'est un phénomène qui se produit couramment lorsque l'on se trouve sous l'eau et que l'on essaye d'observer l'environnement extérieur.



L'autre difficulté réside dans le mélange des couleurs obtenues par les rayons réfléchis et réfractés. On ne peut se contenter de faire une somme, le risque étant que les valeurs soient hors champs et saturent (se maximise à 1.0 ou plus pour nous).

La solution est un outil mathématique décrit par Augustin-Jean Fresnel que l'on appelle couramment l'équation de Fresnel. L'équation de Fresnel permet de calculer la part de réflexion et de réfraction perçue par un observateur en fonction de son orientation par rapport à la surface. Cette équation étant assez complexe – au sens propre, comme au sens mathématique - il est courant en infographie d'avoir recours à l'approximation développée par Christopher Schlick qui est plus adaptée aux espaces colorimétriques RGB.

Fresnel(direction, normal, ior)

Cette approximation dite Fresnel-Schlick nous permet de calculer un facteur de contribution entre nos deux rayons (lerp).

```
Kf = Fresnel(ray.direction, hit.normal, ior)
Color = reflexion * Kf + refraction * (1 - Kf)
```

Références

Généralités

<https://raytracing.github.io/books/RayTracingInOneWeekend.html>

<https://www.realtimerendering.com/raytracing/An-Introduction-to-Ray-Tracing-The-Morgan-Kaufmann-Series-in-Computer-Graphics-.pdf>

Intersections

<https://www.scratchapixel.com/lessons/3d-basic-rendering/minimal-ray-tracer-rendering-simple-shapes/ray-sphere-intersection>

<https://www.scratchapixel.com/lessons/3d-basic-rendering/minimal-ray-tracer-rendering-simple-shapes/ray-plane-and-ray-disk-intersection>

<https://www.scratchapixel.com/lessons/3d-basic-rendering/ray-tracing-rendering-a-triangle/moller-trumbore-ray-triangle-intersection>

Light Transport / Matériaux

<https://www.scratchapixel.com/lessons/3d-basic-rendering/ray-tracing-overview/light-transport-ray-tracing-whitted>

<https://www.scratchapixel.com/lessons/3d-basic-rendering/introduction-to-shading/reflection-refraction-fresnel>